

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME:	ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS	COURSE CODE:	MLA0101
--------------	---	-----------------	---------

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

I Yoga Madha A-192425058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

1.PROLOG-BASED VEHICLE FAULT DIAGNOSIS EXPERT SYSTEM

Abstract

The Prolog-Based Vehicle Fault Diagnosis Expert System is a rule-based artificial intelligence system designed to assist vehicle service centers in identifying probable vehicle faults from observed symptoms. The system uses a knowledge base containing vehicle symptoms, production rules, and recommended diagnostic actions. Prolog is used to represent knowledge and perform logical inference.

The system supports forward chaining and backward chaining to demonstrate different reasoning approaches. It also demonstrates important Prolog concepts such as unification and backtracking. Based on the symptoms entered by the user, the system identifies a probable fault and provides a suitable diagnostic recommendation.

The system can be used as a preliminary decision-support tool in automobile service centers. It provides explainable results because the rules used to reach a conclusion can be inspected.

Introduction

The automobile industry depends heavily on vehicle maintenance and fault diagnosis. A vehicle may show different symptoms such as engine overheating, difficulty starting, abnormal engine noise, low mileage, warning-light activation, braking problems, or poor acceleration.

Traditional diagnosis generally depends on the experience of a mechanic. A rule-based expert system can support this process by storing expert knowledge as facts and production rules.

In this project, Prolog is used to develop an expert system for vehicle fault diagnosis. The system receives vehicle symptoms as input, compares them with predefined production rules, identifies a probable fault, and recommends an appropriate diagnostic action.

The project follows the automobile problem given in the assignment, which requires a Prolog-based expert system to identify probable vehicle faults and recommend diagnostic actions using production rules and logical inference.

Problem Statement and Objectives

Problem Statement

A vehicle service center receives customer complaints such as:

- Engine overheating
- Difficulty starting
- Abnormal engine noise
- Low mileage
- Warning-light activation
- Poor acceleration
- Brake problems
- Battery problems
- Transmission problems

It is necessary to develop an expert system that can analyze these symptoms and identify probable vehicle faults.

The system should also recommend suitable diagnostic actions and explain the reasoning behind the result.

Inputs

The system accepts vehicle symptoms such as:

- High temperature
- Low coolant
- Coolant leakage
- Difficulty starting
- Dim headlights
- Clicking noise
- Engine noise
- Low mileage
- Poor acceleration
- Soft brake pedal
- Warning lights
- Rough engine
- Low tyre pressure
- Gear shifting difficulty

Expected Outputs

The system produces:

- Probable vehicle fault
- Required symptoms
- Recommended diagnostic action
- Explanation of the reasoning

Stakeholders

The main stakeholders are:

- Vehicle service centers
- Automobile mechanics
- Service advisors
- Vehicle owners
- Diagnostic technicians

Objectives

The main objectives are:

1. Develop a Prolog-based automobile expert system.
2. Represent vehicle knowledge using facts and production rules.
3. Identify probable vehicle faults from symptoms.
4. Implement forward chaining.
5. Implement backward chaining.
6. Demonstrate unification.
7. Demonstrate backtracking.
8. Provide explanations for diagnostic decisions.
9. Test the system using realistic vehicle scenarios.
10. Provide appropriate diagnostic recommendations.

Domain Analysis and Knowledge Acquisition

Automobile Domain

The automobile domain contains different vehicle components such as:

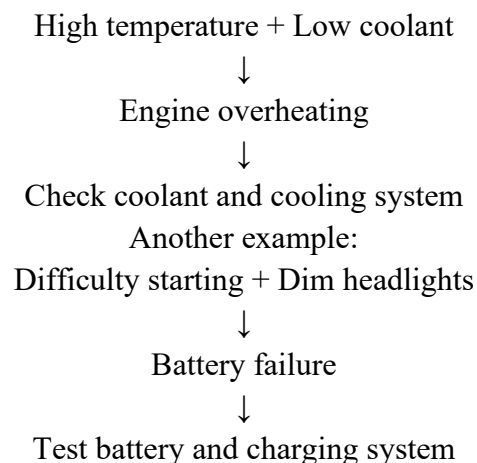
- Engine
- Battery
- Starter motor
- Cooling system
- Fuel system
- Brake system
- Tyres
- Alternator
- Transmission
- Ignition system

Each component can develop faults that produce observable symptoms.

Knowledge Acquisition

Knowledge for the expert system is represented as relationships between vehicle symptoms, probable faults, and recommended actions.

For example:



Entities

Important entities include:

- Vehicle
- Symptom
- Fault
- Vehicle component
- Diagnostic action
- Rule

Facts

Examples of facts are:

symptom(high_temperature).

symptom(low_coolant).

symptom(difficulty_starting).

symptom(dim_headlights).

symptom(poor_acceleration).

Conditions

Conditions represent symptoms that must be satisfied before a rule can be applied.

For example:

high_temperature

low_coolant

Decisions

The system decides the probable vehicle fault based on the available symptoms.

Knowledge Representation and Production Rules

Knowledge is represented using Prolog facts, predicates, and production rules.

Basic Fact

symptom(high_temperature).

This represents the fact that high temperature is a recognized vehicle symptom.

Production Rule

A production rule follows the basic structure:

IF conditions

THEN conclusion

For example:

```
vehicle_rule(  
    engine_overheating,  
    [high_temperature, low_coolant],  
    'Check coolant level, radiator and cooling system.'  
).
```

This means:

IF high temperature

AND low coolant

THEN engine overheating.

Example Rules

Engine Overheating

```
vehicle_rule(  
    engine_overheating,  
    [high_temperature, low_coolant],  
    'Check coolant level, radiator and cooling system.'  
).
```

Battery Failure

```
vehicle_rule(  
    battery_failure,  
    [difficulty_starting, dim_headlights],  
    'Test battery voltage and charging system.'  
).
```

Brake Problem

```
vehicle_rule(  
    brake_problem,  
    [brake_warning_light, soft_brake_pedal],  
    'Inspect brake fluid, brake lines and brake pads.'  
).
```

Engine Misfire

```
vehicle_rule(  
    engine_misfire,  
    [rough_engine, poor_acceleration],  
    'Inspect spark plugs, ignition coils and fuel injectors.'  
).
```

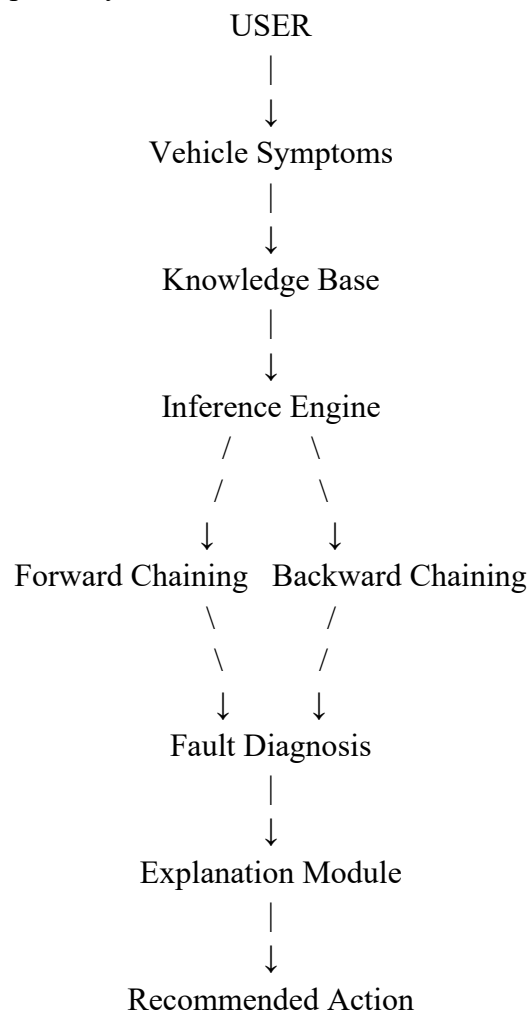
Transmission Problem

```
vehicle_rule(  
    transmission_problem,  
    [gear_shifting_difficulty, abnormal_transmission_noise],  
    'Inspect transmission fluid and transmission components.'  
).
```

The knowledge representation uses facts and production rules as required for the Prolog expert system.

Expert System Architecture

The architecture of the proposed system is:



Components

User

The user enters the observed vehicle symptoms.

Knowledge Base

The knowledge base stores:

- Vehicle symptoms
- Faults
- Production rules
- Diagnostic actions

Inference Engine

The inference engine applies logical reasoning to identify a probable fault.

It supports:

- Forward chaining
- Backward chaining
- Unification
- Backtracking

Explanation Module

The explanation module provides the reason behind the identified fault.

Output

The final output contains:

Probable Fault

Recommended Action

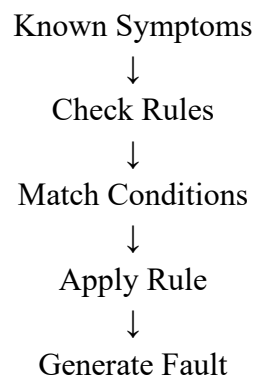
Reasoning

Inference Mechanism and Algorithms

Forward Chaining

Forward chaining starts from the known symptoms and moves toward a conclusion.

Process



Example

Given:

high_temperature

low_coolant

The system finds:

IF high_temperature AND low_coolant

THEN engine_overheating

Therefore:

Fault = engine_overheating

Prolog Query

?- forward_diagnosis(

```
    [high_temperature, low_coolant],  
    Fault  
).
```

Output:

Fault = engine_overheating.

Backward Chaining

Backward chaining starts with a possible goal or fault and works backward to determine the conditions required to prove it.

Process

Possible Fault



Find Rule



Find Required Conditions



Check Symptoms



Confirm Fault

Example

Goal:

engine_overheating

Required conditions:

high_temperature

low_coolant

Prolog Query

```
?- backward_diagnosis(  
    engine_overheating,  
    Symptoms  
).
```

Output:

Symptoms = [high_temperature, low_coolant].

The assignment specifically requires demonstration of both forward and backward chaining.

Unification

Unification is the process of matching Prolog terms and variables.

Example:

vehicle_rule(Fault, Symptoms, Action).

Prolog can unify the variables with information stored in the knowledge base.

Example result:

Fault = engine_overheating

Symptoms = [high_temperature, low_coolant]

Action = 'Check coolant level, radiator and cooling system.'

Thus, unification allows Prolog to match a query with a suitable rule.

Backtracking

Backtracking is an important Prolog mechanism.

If more than one rule can match the available symptoms, Prolog can return to an earlier decision point and try another possible rule.

Example:

```
list_possible_faults(  
    [high_temperature, low_coolant]  
).
```

The system searches the knowledge base for matching rules.

If one possible rule does not produce the required result, Prolog can backtrack and search for another matching rule.

Prolog Implementation

Software Used

The project is implemented using:

Programming Language : Prolog

IDE : Visual Studio Code

Prolog Environment : SWI-Prolog

Version Control : GitHub

Main Files

main.pl

knowledge_base.pl

inference_engine.pl

explanation.pl

test_cases.pl

forward_backward_demo.pl

README.md

Important Predicate

The main rule predicate is:

vehicle_rule(Fault, RequiredSymptoms, Action).

It represents:

Fault

Required Symptoms

Recommended Action

Diagnosis Predicate

diagnose(Symptoms, Fault, Action).

This predicate receives the symptoms and produces the diagnosis.

Example Query

```
?- diagnose(  
    [high_temperature, low_coolant],  
    Fault,  
    Action
```

).

Output:

Fault = engine_overheating

Action =

'Check coolant level, radiator and cooling system.'

Loading the Program

Open SWI-Prolog and enter:

?- [main].

Then:

?- start.

The system asks the user to enter symptoms.

Example:

Symptom: high_temperature.

Symptom: low_coolant.

Symptom: done.

Output:

Observed Symptoms:

[high_temperature, low_coolant]

Probable Fault:

engine_overheating

Recommended Action:

Check coolant level, radiator and cooling system.

Test Cases and Results

The assignment requires a minimum of five industry-based test cases. Ten test cases are provided in this project.

Test Case 1 – Engine Overheating

Input

[high_temperature, low_coolant]

Query

?- diagnose(

[high_temperature, low_coolant],

Fault,

Action

).

Expected Output

Fault = engine_overheating

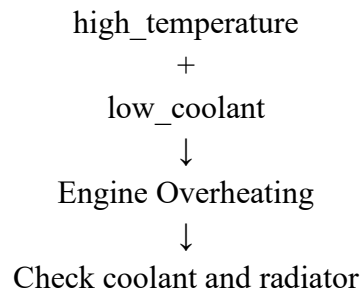
Actual Output

Fault = engine_overheating

Action =

'Check coolant level, radiator and cooling system.'

Inference Trace



Test Case 2 – Battery Failure

Input

[difficulty_starting, dim_headlights]

Expected Output

battery_failure

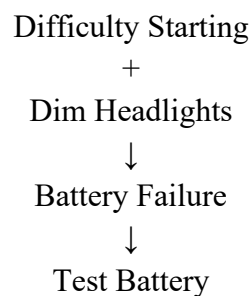
Actual Output

Fault = battery_failure

Action =

'Test battery voltage and charging system.'

Inference Trace



Test Case 3 – Low Engine Oil

Input

[oil_warning_light, engine_noise]

Expected Output

low_engine_oil

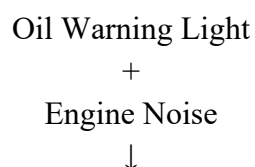
Actual Output

Fault = low_engine_oil

Action =

'Check engine oil level and inspect for oil leakage.'

Inference Trace



Low Engine Oil
↓
Check Engine Oil

Test Case 4 – Fuel System Problem

Input

[low_mileage, poor_acceleration]

Expected Output

fuel_system_problem

Actual Output

Fault = fuel_system_problem

Action =

'Inspect fuel filter, injectors and fuel pressure.'

Inference Trace

Low Mileage
+
Poor Acceleration
↓
Fuel System Problem
↓
Inspect Fuel System

Test Case 5 – Brake Problem

Input

[brake_warning_light, soft_brake_pedal]

Expected Output

brake_problem

Actual Output

Fault = brake_problem

Action =

'Inspect brake fluid, brake lines and brake pads.'

Inference Trace

Brake Warning Light
+
Soft Brake Pedal
↓
Brake Problem
↓
Inspect Brake System

Test Case 6 – Alternator Failure

Input

[battery_warning_light, dim_headlights]

Expected Output

alternator_failure

Actual Output

Fault = alternator_failure

Action =

'Test alternator output and charging circuit.'

Test Case 7 – Engine Misfire

Input

[rough_engine, poor_acceleration]

Expected Output

engine_misfire

Actual Output

Fault = engine_misfire

Action =

'Inspect spark plugs, ignition coils and fuel injectors.'

Test Case 8 – Check Engine Fault

Input

[check_engine_light, rough_engine]

Expected Output

check_engine_fault

Actual Output

Fault = check_engine_fault

Action =

'Scan the vehicle ECU and inspect engine sensors.'

Test Case 9 – Transmission Problem

Input

[gear_shifting_difficulty,
abnormal_transmission_noise]

Expected Output

transmission_problem

Actual Output

Fault = transmission_problem

Action =

'Inspect transmission fluid and transmission components.'

Test Case 10 – Tyre Pressure Problem

Input

[uneven_tyre_wear, low_tyre_pressure]

Expected Output

tyre_pressure_problem

Actual Output

Fault = tyre_pressure_problem

Action =

'Check and correct tyre pressure and inspect tyre condition.'

Analysis, Limitations and Future Enhancements**Analysis**

The developed expert system successfully demonstrates rule-based reasoning for automobile fault diagnosis.

The system can:

- Accept vehicle symptoms.
- Match symptoms with production rules.
- Identify probable faults.
- Recommend diagnostic actions.
- Demonstrate forward chaining.
- Demonstrate backward chaining.
- Demonstrate unification.
- Demonstrate backtracking.
- Provide explainable reasoning.

The use of Prolog makes the system suitable for representing logical relationships between symptoms and faults.

Reasoning Accuracy

The system produces the expected result when the input symptoms satisfy a predefined rule.

For example:

high_temperature + low_coolant

matches:

engine_overheating

Therefore, the system can provide a consistent result for the defined knowledge base.

Limitations

The system has some limitations:

1. It depends on predefined rules.
2. It cannot diagnose faults that are not included in the knowledge base.
3. It does not directly obtain information from vehicle sensors.
4. Multiple faults may sometimes produce similar symptoms.
5. The system is intended for preliminary decision support and not as a replacement for professional vehicle inspection.
6. The current system uses manually entered symptoms.

Future Enhancements

The system can be improved by adding:

- Web-based user interface
- Mobile application
- Database integration
- OBD sensor integration
- Real-time vehicle monitoring
- Machine learning support
- Voice-based symptom input
- Automatic service-center notification
- Fault severity classification
- Historical vehicle maintenance records

Conclusion

The Prolog-Based Vehicle Fault Diagnosis Expert System demonstrates the application of artificial intelligence and expert-system techniques in the automobile industry.

The system uses a knowledge base containing vehicle symptoms, production rules, faults, and diagnostic actions. Prolog provides logical inference through rule matching, unification, and backtracking.

Both forward chaining and backward chaining are demonstrated. Forward chaining starts from observed symptoms and derives a probable fault, while backward chaining starts from a possible fault and identifies the conditions required to establish it.

The system was tested using multiple realistic vehicle scenarios, including overheating, battery failure, engine-oil problems, fuel-system problems, brake problems, alternator failure, engine misfire, transmission problems, and tyre-pressure problems.

The developed system provides a simple, explainable and practical foundation for automobile fault diagnosis. It can be further enhanced using vehicle sensor data, databases, web interfaces and machine-learning techniques.

References

1. SWI-Prolog Documentation, SWI-Prolog Reference Manual.
2. Ivan Bratko, *Prolog Programming for Artificial Intelligence*, Pearson.
3. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
4. George F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*.
5. Course material: Artificial Intelligence and Expert Systems, SIMATS Engineering.
6. Assignment specification: Industry Problem-Based Assignment, Artificial Intelligence and Expert Systems.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference traces.	7–8% – Provides relevant test cases with mostly accurate results.	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation, GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements.	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				